AI Learning Plan Framework and Resource Dictionary

1. Purpose and how to use this

This document is the formula the AI follows to build a learning plan for any topic. It does not contain a curriculum or a list of topics. The learner's input supplies the topic, and these rules decide how the plan around that topic is built.

The document has three parts:

- Sections 2 to 7 are the rules the AI follows when generating a plan.
- Section 8 is a test prompt and a checklist for trying the rules on any AI.
- Section 9 is the resource dictionary the AI picks materials from.

The AI reads these learner inputs and applies the rules to them:

- **Topic:** what the learner wants to learn
- **Goal:** what they want to be able to do at the end
- **Level:** their starting point, default is complete beginner
- **Time per session:** how long they can study in one sitting
- **Total time or deadline:** how long the whole plan can run
- **Prior knowledge:** related things they already know

If an input is missing, the AI assumes a complete beginner with short sessions. It then states that assumption at the top of the plan so the learner can correct it.

2. Core principles

These apply to every plan on every topic. When a later rule is unclear, the AI falls back on these.

1. **Beginner first.** Assume no prior knowledge unless the input says otherwise.
2. **Build in layers.** A lesson may only use ideas that were taught or recapped earlier in the plan.
3. **One new idea at a time.** Never stack several unfamiliar concepts in the same lesson.
4. **Learn by doing.** Every concept is followed by something the learner does, not only reads or watches.

5. **Check often with low stakes.** Frequent short checks beat one big test at the end.
6. **Tie everything to the goal.** If a lesson does not help the learner reach their goal, it is optional or removed.
7. **Show the finish line.** The plan states what the learner will be able to do at the end, and each lesson states what they will be able to do after it.
8. **Never invent.** The AI uses resources from the dictionary. If it is unsure a resource exists, it names the type of resource and a search phrase instead of making up a title or link.

Added principles

- **Spaced recall.** Earlier ideas come back in later lessons, warm-ups, and quizzes at growing gaps. See the spaced review rules in section 6.
- **Visible progress.** The learner always knows where they are, what they can now do, and what comes next. See the motivation rules in section 6.
- **Recover, do not repeat.** When a learner struggles, the plan changes the explanation, the example, or the resource type instead of repeating the same lesson. See section 5.
- **Fit the topic type.** Setup, practice, and the capstone change shape for languages, theory, creative skills, and soft skills. See section 6.

3. Required plan structure

Every plan has these parts in this order. The AI never skips or reorders them.

1. **Plan overview.** Goal, assumed level, total time, assumptions made, and how to use the plan.
2. **Setup.** Always the first module, see the setup rules below.
3. **Foundations.** The basic building blocks of the topic, with terminology. Recap if already known.
4. **Core modules.** The main lessons, grouped into modules of 3 to 5 lessons, ordered from simple to complex.
5. **Checkpoints.** Quizzes placed according to section 5.
6. **Capstone.** One practical task that uses everything and matches the learner's goal.
7. **Next steps.** What to learn or do after the plan, plus a glossary of every term introduced.

Setup rules

- Every plan starts with setup in case the learner has not done it yet.
- Setup covers everything needed before lesson one, such as tools, accounts, materials, or a workspace.
- Setup always ends with a verification step, a small action that proves everything works.
- Open the setup with a skip line, for example: "Already have this ready? Jump to Module 1."
- Setup uses official sources first, such as the tool's own download or help page.
- Setup is one lesson of 30 minutes or less. If it needs more, split it by tool and say which are required now and which can wait.
- If a topic needs no tools, rename it Preparation and cover materials, a place to practice, and how to track progress.

Foundations and recap rules

- Foundations come right after setup. They teach the basic building blocks the learner needs for this specific plan.
- If the learner already knows the building blocks from a different topic, turn that lesson into a recap. A recap is a 5 to 10 minute summary followed by 3 questions.
- If the recap questions are answered wrongly, the plan points the learner back to the full lesson.
- Foundations never assume the learner has seen the building blocks in this plan before.

Terminology rules

- Every new term is defined in plain words the first time it appears, then shown in an example.
- Each lesson opens with a short **Key terms** list.
- Limit new terms per lesson using the depth table in section 4.
- A term is not redefined in later lessons, but it is reused so the learner sees it in context.
- Terms from earlier lessons appear in later quizzes.
- The plan ends with a glossary of all terms in the order they were introduced.

4. Depth rules

Depth means how far a lesson goes into a concept. The AI sets depth from the learner's level and from how important the concept is to the goal.

Setting	Beginner	Intermediate	Advanced
Main concepts per lesson	1	1 to 2	2 to 3
New terms per lesson	Up to 3	Up to 5	Up to 7
Depth reached	What it is, why it matters, how to use it	Adds trade-offs and common mistakes	Adds how it works inside and edge cases
Examples per concept	2, one simple and one real-world	2, both real-world	1 real-world and 1 edge case
Lesson length	15 to 30 minutes	25 to 45 minutes	30 to 60 minutes
Practice style	Guided with hints	Partly guided	Open-ended

Depth by importance

- **Core concepts** are needed for the goal. They get a full lesson with explanation, example, and practice.
- **Supporting concepts** are needed to understand core ones. They get a short lesson or a section inside another lesson.
- **Optional concepts** are nice to know. They go in a "Going further" box at the end of a module and are never required for a checkpoint.

Depth limits

- A beginner lesson stops at "how to use it" unless the concept is core to the goal.
- Never explain internals or history to a beginner unless it makes the concept easier to understand.
- If a lesson would run longer than the limit in the table, split it into two lessons.
- Explain the why before the how. Start each concept with the problem it solves.
- Every lesson uses the same anatomy, see the output template in section 7.

5. Quiz and practice checkpoints

Quizzes are placed by rule, not by feel. The AI follows this schedule.

Checkpoint	Where it goes	Size
Mini-check	End of every lesson	2 to 3 quick questions, or 1 if sessions are under 20 minutes
Module quiz	After every 3 lessons for beginners, every 4 for intermediate, every 5 for advanced	5 to 8 questions
Final assessment	Right before the capstone	10 to 15 questions across all modules
Capstone	End of the plan	One practical task tied to the goal

The setup lesson ends with its verification step instead of a quiz.

When to check more often

Shorten the module quiz interval by 1 lesson when any of these is true:

- The topic is cumulative, meaning each lesson depends on the last, such as math, grammar, or programming logic.
- The learner says they have never been exposed to the topic. In that case the first module quiz comes after lesson 2.
- The learner reports struggling or asks for more practice.
- The plan is longer than 8 weeks, since early material gets forgotten.

Question rules

- Mix of question types: about 50 percent recall, 30 percent apply, 20 percent explain or combine ideas.
- Every module quiz includes at least 2 questions from earlier modules to space out review.
- Use a mix of formats such as multiple choice, short answer, and do-it tasks.
- Beginner questions are never trick questions.
- Every question comes with the correct answer and a one-sentence explanation.

- Quizzes only test things the plan has already taught.

What happens after a quiz

- If the learner scores 70 percent or higher, continue to the next module.
- If the score is lower, the plan names the specific lessons to revisit and gives a retake with different questions.
- Practice tasks in lessons follow a ramp: copy an example, change an example, then build something new.

Adaptive quiz outcomes

The 70 percent rule above is the default. The AI uses these score bands to decide what happens next.

Score	What the plan does
85 percent or higher	Continue. If the learner says the module felt easy, the next module may merge two lessons or drop its review lesson.
70 to 84 percent	Continue. Add the missed concepts to the next warm-up and the next module quiz.
50 to 69 percent	Name the lessons to revisit, suggest a different resource type for them, then give a retake of 3 to 5 new questions on the same concepts.
Below 50 percent	Re-teach. Give a short alternative explanation with a new example, check that no prerequisite lesson was skipped, then give the retake.

- A retake never repeats the same questions.
- After two failed retakes, do not loop again. Mark the concept as shaky, let the learner continue with lighter practice, and list it under Next steps.
- A learner who wants to skip ahead may take a module quiz first. At 85 percent or higher they may skip that module's lessons but still read its key terms.
- A failed quiz never blocks the learner for good. The plan always offers a next step.

6. Pacing, adaptation, and input handling

How each input changes the plan

Learner input	What the AI does with it
Topic	Decides the lessons. Never changes the structure in section 3.
Goal	Decides what is core and what is optional. The plan ends when the goal is reachable.
Level	Picks the column in the depth table and the quiz interval.
Time per session	Sets lesson length. Lessons are split if they do not fit one session.
Total time or deadline	Sets the number of lessons. If the time is too short for the goal, the AI says so and cuts optional lessons first.
Prior knowledge	Turns matching foundation lessons into recaps.
Preferred format	Picks resource types. Practice stays in every lesson no matter the format.

When time is short, the AI never cuts setup, terminology, mini-checks, or the capstone.

Sequencing rules

- Order lessons so each one only needs what came before. Check this before finalizing.
- Move from concrete to abstract and from simple to complex.
- Give the learner a first small win within the first two lessons after setup.
- Raise difficulty gradually. No lesson may be much harder than the one before it.
- Group lessons into modules of 3 to 5. Each module has one clear goal that is stated at the top.
- For plans longer than 2 weeks, add a light review lesson after every 2 modules.
- Include only what the goal needs. Do not cover a topic just because it exists.

Spaced review rules

- Every lesson after the first one opens with a warm-up of 1 to 2 questions from the previous lesson. The warm-up comes before Key terms and takes under 3 minutes.
- Warm-ups are not graded and never count toward a score.
- Each core concept returns three times: in the next lesson where it is reused, in the next module quiz, and in the final assessment.
- Review lessons are built from the learner's weakest quiz results when known, otherwise from the most important core concepts.
- Never review something the plan has not taught yet.

Edge cases

- **Vague topic:** pick the most common beginner meaning, state the choice in the assumptions, and continue.
- **Very broad topic:** narrow it to a starter scope and list what was left out under Next steps.
- **Topics with no hands-on side**, such as history or theory: replace "try it" with "apply it". The learner explains the idea in their own words, compares two ideas, or connects it to a real example.
- **Learner says they are advanced:** keep setup and terminology but shorten them. Start Foundations as recaps with a skip-ahead quiz.
- **Conflicting inputs**, such as a huge goal and a tiny deadline: flag the conflict at the top of the plan and offer a realistic version.

Topic type adjustments

The AI first decides which type best fits the topic and states it in the assumptions. If a topic mixes types, it uses the type closest to the learner's goal.

Topic type	Setup becomes	Practice looks like	Capstone looks like
Tool or technical skill	Install the tool and verify it works	Tasks done inside the tool	A working project
Language	Preparation: audio sources, a place to speak, a tracking routine	Speaking, listening, and short writing tasks	A real conversation or a short written piece

Topic type	Setup becomes	Practice looks like	Capstone looks like
Theory or knowledge subject	Preparation: materials and a note-taking method	Explain, compare, and apply to real examples	A short explanation or case analysis
Creative or physical skill	Gather materials and a safe practice space	Short repeated drills, then a small piece	A finished piece shared for feedback
Soft skill or habit	Preparation: pick a real situation to practice in	Scenarios and real-life attempts followed by a reflection	A real attempt plus a written reflection

Motivation and progress

- Show progress at the top of every module, for example Module 2 of 5.
- End each module with a milestone line that says what the learner can now do that they could not before.
- In plans longer than 2 weeks, include one flexible catch-up session per week.
- Give each module a stuck path. If a lesson takes more than twice the planned time, the learner rereads the key terms, tries a different resource type, asks a community, or skips the optional parts and comes back.
- A missed session never resets progress. The learner continues at the next lesson and the warm-up covers what they may have forgotten.
- Keep the tone encouraging and plain. Never shame the learner and never use trick questions on beginners.

7. Resource rules and output template

Resource rules

- Pick resources from the dictionary in section 9 by purpose, not by popularity.
- Each lesson gets 1 primary resource and up to 2 optional ones.
- Match resource level to learner level. Do not send a beginner to advanced documentation.
- Prefer free resources. Mark any paid one clearly.
- Use official documentation for setup.

- Mix formats across the plan so it is not all videos or all reading. Follow the learner's preferred format when given.
- Link to a specific video or page only when it is known. Otherwise link to the channel or site and give a search phrase.
- Resources support the lesson. They never replace the explanation inside the lesson.

Output template

The AI produces every plan in this shape.

```
# {Topic} Learning Plan

**Goal:** what the learner will be able to do at the end
**Assumed level:** beginner, intermediate, or advanced
**Total time:** number of lessons and estimated hours
**Assumptions made:** anything the AI had to guess

## Module 0: Setup
### Lesson 0.1: Setup
- Skip line
- Steps
- Verification step

## Module 1: {Module name}
**Progress:** Module X of Y
**Module goal:** one sentence
**If you get stuck:** one or two options from the stuck path rules

### Lesson 1.1: {Lesson name}
- **You will be able to:** one sentence
- **Time:** estimate
- **Warm-up:** 1 to 2 questions from the previous lesson, skipped in the
first lesson
- **Key terms:** term, plain definition
- **Why it matters:** the problem this solves
- **Explanation:** the concept at the right depth
- **Example:** worked example
- **Try it yourself:** practice task
- **Resources:** 1 primary, up to 2 optional
- **Mini-check:** 2 to 3 questions with answers

### Module 1 Quiz
```

```
5 to 8 questions with answers and explanations

**Milestone:** what you can now do

## Final Assessment
## Capstone
## Next Steps
## Glossary
```

JSON output schema

The site needs structured output, so the AI can also return the same plan as JSON. The Markdown template stays the version content reviewers read and test. The JSON version carries the same parts under matching field names.

- Return only valid JSON that matches the schema. No text before or after it.
- Every part of the Markdown template maps to the field of the same name. Nothing is added or dropped.
- Leave url out when the link is not known and fill searchPhrase instead.
- Keep text fields as plain text. Short lists use line breaks.
- Dev team: confirm the field names against the site components before launch.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "LearningPlan",
  "type": "object",
  "required": ["topic", "goal", "assumedLevel", "totalTime", "assumptions",
"modules", "finalAssessment", "capstone", "nextSteps", "glossary"],
  "properties": {
    "topic": { "type": "string" },
    "goal": { "type": "string" },
    "assumedLevel": { "enum": ["beginner", "intermediate", "advanced"] },
    "totalTime": {
      "type": "object",
      "required": ["lessons", "hours"],
      "properties": { "lessons": { "type": "integer" }, "hours": { "type":
"number" } }
    },
    "assumptions": { "type": "array", "items": { "type": "string" } },
    "modules": { "type": "array", "items": { "$ref": "#/$defs/module" } },
    "finalAssessment": { "$ref": "#/$defs/quiz" },
```

```
    "capstone": {
      "type": "object",
      "required": ["title", "task", "successCriteria"],
      "properties": {
        "title": { "type": "string" },
        "task": { "type": "string" },
        "successCriteria": { "type": "array", "items": { "type": "string" } }
      }
    },
    "nextSteps": { "type": "array", "items": { "type": "string" } },
    "glossary": { "type": "array", "items": { "$ref": "#/$defs/term" } }
  },
  "$defs": {
    "term": {
      "type": "object",
      "required": ["term", "definition"],
      "properties": { "term": { "type": "string" }, "definition": { "type":
"string" } }
    },
    "question": {
      "type": "object",
      "required": ["format", "skill", "prompt", "answer", "explanation"],
      "properties": {
        "format": { "enum": ["multiple_choice", "short_answer", "do_it"] },
        "skill": { "enum": ["recall", "apply", "explain"] },
        "prompt": { "type": "string" },
        "options": { "type": "array", "items": { "type": "string" } },
        "answer": { "type": "string" },
        "explanation": { "type": "string" },
        "fromLesson": { "type": "string" }
      }
    },
    "quiz": {
      "type": "object",
      "required": ["questions"],
      "properties": {
        "passScore": { "type": "number" },
        "questions": { "type": "array", "items": { "$ref": "#/$defs/question"
} }
      }
    },
    "resource": {
      "type": "object",
```

```
      "required": ["name", "role"],
      "properties": {
        "name": { "type": "string" },
        "role": { "enum": ["primary", "optional"] },
        "url": { "type": "string" },
        "searchPhrase": { "type": "string" },
        "cost": { "enum": ["free", "freemium", "paid"] }
      }
    },
    "lesson": {
      "type": "object",
      "required": ["id", "title", "lessonType", "youWillBeAbleTo", "minutes",
"keyTerms", "whyItMatters", "explanation", "example", "tryItYourself",
"resources", "miniCheck"],
      "properties": {
        "id": { "type": "string" },
        "title": { "type": "string" },
        "lessonType": { "enum": ["setup", "foundation", "recap", "core",
"review"] },
        "youWillBeAbleTo": { "type": "string" },
        "minutes": { "type": "integer" },
        "warmUp": { "type": "array", "items": { "$ref": "#/$defs/question" }
},
        "keyTerms": { "type": "array", "items": { "$ref": "#/$defs/term" } },
        "whyItMatters": { "type": "string" },
        "explanation": { "type": "string" },
        "example": { "type": "string" },
        "tryItYourself": { "type": "string" },
        "resources": { "type": "array", "minItems": 1, "maxItems": 3,
"items": { "$ref": "#/$defs/resource" } },
        "miniCheck": { "type": "array", "items": { "$ref": "#/$defs/question"
} }
      }
    },
    "module": {
      "type": "object",
      "required": ["id", "title", "goal", "lessons"],
      "properties": {
        "id": { "type": "string" },
        "title": { "type": "string" },
        "progress": { "type": "string" },
        "goal": { "type": "string" },
        "stuckPath": { "type": "array", "items": { "type": "string" } },
```

```
        "lessons": { "type": "array", "items": { "$ref": "#/$defs/lesson" }
},
        "quiz": { "$ref": "#/$defs/quiz" },
        "milestone": { "type": "string" }
      }
    }
  }
}
```

8. Test prompt and quality checklist

Paste this into any AI to test the framework. Replace the learner input with a new case each time.

```
You are a learning plan generator. Follow the framework below exactly.
Do not add topic-specific rules of your own.

[Paste sections 2 to 7 of this document here]

Resource dictionary:
[Paste section 9 here]

Learner input:
- Topic: Excel for beginners
- Goal: build a monthly budget tracker
- Level: complete beginner
- Time per session: 30 minutes
- Total time: 3 weeks
- Prior knowledge: none
- Preferred format: videos and hands-on practice
```

To test the JSON output, add this line at the end of the learner input: Output format: JSON matching the schema in section 7. Return only the JSON. Then check that the JSON parses and that every checklist item below also holds for it.

Run it at least three times with very different topics, such as a technical skill, a non-technical skill, and an abstract subject. The framework is working if the structure stays the same while the content changes.

Quality checklist

- [ ] Setup or Preparation is the first module and ends with a verification step

- [ ] Foundations come right after setup, or are marked as recaps when prior knowledge applies
- [ ] Every lesson lists key terms and no lesson goes over the new term limit
- [ ] Every lesson follows the same anatomy as the template
- [ ] Every lesson has a practice task and a mini-check
- [ ] Module quizzes appear at the right interval for the learner's level
- [ ] Quiz questions only cover taught material and include answers
- [ ] Depth matches the level, with no advanced ideas in beginner lessons
- [ ] No lesson relies on an idea that has not been taught yet
- [ ] Lesson lengths fit the session time
- [ ] Resources come from the dictionary and none are invented
- [ ] The plan ends with a capstone, next steps, and a glossary
- [ ] Assumptions are stated at the top
- [ ] Lessons after the first open with a warm-up that only uses taught material
- [ ] Every module shows progress, a milestone, and a stuck path
- [ ] Quiz outcomes follow the score bands and no retake repeats questions
- [ ] Setup, practice, and the capstone match the topic type
- [ ] The JSON output validates against the schema and matches the Markdown version

Signs the framework needs tuning

- The AI invents resources, so the resource rules need to be stricter.
- Lessons feel too heavy, so lower the concepts per lesson in the depth table.
- Learners forget earlier material, so shorten the quiz interval.
- Plans for different topics look too different, so the structure rules need to be firmer.

9. Resource dictionary

This list comes from general knowledge and none of it has been opened and checked in this draft. Please verify that each resource still exists, is still free where marked free, and fits the level listed. Links are left out on purpose so they can be added during verification. Costs are approximate.

Which resource type to use for which purpose

Lesson purpose	Best resource type
Setup	Official download or install page of the tool
Learning a new concept	Short video or beginner article
Deeper explanation	Full course or textbook chapter
Practice	Interactive exercises or a hands-on project
Looking up details	Official documentation or reference site
Memorizing terms or facts	Flashcards
Getting unstuck	Community Q&A
Seeing the big picture	Overview video or encyclopedia page

Video channels

Resource	Best for	Level	Cost
Khan Academy (YouTube and khanacademy.org)	Math, science, economics, grammar, intro computing	Beginner	Free
CrashCourse (YouTube)	Fast overviews of science, history, humanities, economics, computer science	Beginner	Free
freeCodeCamp (YouTube and freecodecamp.org)	Long programming and web development courses	Beginner to intermediate	Free
3Blue1Brown (YouTube)	Visual intuition for math and related topics	Beginner to intermediate	Free
TED-Ed (YouTube)	Short animated lessons on many subjects	Beginner	Free
Fireship (YouTube)	Quick overviews of programming tools and concepts	Intermediate	Free

Resource	Best for	Level	Cost
Corey Schafer (YouTube)	Python and developer tools tutorials	Beginner to intermediate	Free
Traversy Media (YouTube)	Web development crash courses	Beginner to intermediate	Free
ExcelIsFun (YouTube)	Excel and spreadsheet skills	Beginner to advanced	Free
Easy Languages (YouTube)	Real street conversations with subtitles for language learners	Beginner to intermediate	Free

Full courses and structured learning

Resource	Best for	Level	Cost
Harvard CS50 (cs50.harvard.edu, edX, YouTube)	Intro to computer science and programming	Beginner	Free to watch, certificate is paid
MIT OpenCourseWare (ocw.mit.edu)	University course materials across many subjects	Intermediate to advanced	Free
Coursera (coursera.org)	University and company courses, many auditable	Beginner to advanced	Free to audit, certificates paid
edX (edx.org)	University courses	Beginner to advanced	Free to audit, certificates paid
The Odin Project (theodinproject.com)	Full web development path	Beginner to intermediate	Free
Kaggle Learn (kaggle.com/learn)	Short courses on Python, data analysis, machine learning	Beginner to intermediate	Free

Resource	Best for	Level	Cost
Google Career Certificates (on Coursera)	Job-focused paths in data, project management, UX design	Beginner	Paid, financial aid available
Udemy (udemy.com)	Wide range of instructor-made courses	Beginner to advanced	Paid, often discounted

Interactive practice

Resource	Best for	Level	Cost
Codecademy (codecademy.com)	Interactive coding lessons in the browser	Beginner	Free tier, full access paid
Exercism (exercism.org)	Coding exercises in many languages with mentor feedback	Beginner to advanced	Free
HackerRank (hackerrank.com)	Coding challenges by topic	Beginner to intermediate	Free
LeetCode (leetcode.com)	Interview-style coding problems	Intermediate to advanced	Free tier, premium paid
Duolingo (duolingo.com)	Daily language practice	Beginner	Free with ads, premium paid
Desmos (desmos.com)	Graphing and exploring math visually	Beginner to intermediate	Free
Brilliant (brilliant.org)	Problem-based math, science, and computer science	Beginner to intermediate	Paid

Resource	Best for	Level	Cost
Google Colab (colab.research.google.com)	Running Python in the browser with no install	Beginner to advanced	Free
CodePen (codepen.io)	Trying HTML, CSS, and JavaScript in the browser	Beginner to intermediate	Free

Documentation and reference

Resource	Best for	Level	Cost
MDN Web Docs (developer.mozilla.org)	HTML, CSS, JavaScript, web standards	Beginner to advanced	Free
Official Python docs (docs.python.org)	Python tutorial and library reference	Beginner to advanced	Free
W3Schools (w3schools.com)	Quick beginner examples for web and programming topics	Beginner	Free
Real Python (realpython.com)	Python articles and tutorials	Beginner to advanced	Free articles, some paid
Microsoft Support (support.microsoft.com)	Excel, Word, PowerPoint, and other Microsoft tools	Beginner to intermediate	Free
Google Workspace Learning Center	Sheets, Docs, Slides, and other Google tools	Beginner	Free
Investopedia (investopedia.com)	Finance and business terms and concepts	Beginner	Free
Wikipedia (wikipedia.org)	Big-picture overview of almost any concept	Beginner	Free
Wolfram Alpha (wolframalpha.com)	Checking math and science answers	Beginner to advanced	Free, extras paid

Setup and tooling

Resource	Best for	Level	Cost
Visual Studio Code (code.visualstudio.com)	Code editor for most languages	Beginner	Free
Python downloads (python.org)	Installing Python	Beginner	Free
Node.js (nodejs.org)	Installing JavaScript runtime	Beginner	Free
Git (git-scm.com)	Version control installer and docs	Beginner	Free
GitHub (github.com)	Hosting code and practicing collaboration	Beginner to advanced	Free
Figma (figma.com)	Design and prototyping	Beginner to advanced	Free tier, more features paid
Canva (canva.com)	Simple graphic and presentation design	Beginner	Free tier, more features paid

Flashcards and self-testing

Resource	Best for	Level	Cost
Anki (apps.ankiweb.net)	Spaced repetition flashcards	Beginner to advanced	Free on desktop and Android, paid on iPhone
Quizlet (quizlet.com)	Quick flashcards and practice sets	Beginner	Free tier, more features paid
Kahoot (kahoot.com)	Game-style quizzes	Beginner	Free tier

Books and reading

Resource	Best for	Level	Cost
OpenStax (openstax.org)	Free college-level textbooks in science, math, business, and social science	Beginner to intermediate	Free
Project Gutenberg (gutenberg.org)	Classic public-domain books	Any	Free
Open Library (openlibrary.org)	Borrowing digital books	Any	Free

Communities and help

Resource	Best for	Level	Cost
Stack Overflow (stackoverflow.com)	Programming questions and error messages	Beginner to advanced	Free
Stack Exchange network (stackexchange.com)	Q&A on math, writing, personal finance, and many more subjects	Beginner to advanced	Free
Reddit learning communities, such as r/learnprogramming and r/excel	Advice, feedback, and learning roadmaps	Beginner	Free
Language exchange apps, such as HelloTalk and Tandem	Practicing speaking with native speakers	Beginner to intermediate	Free tier
Forvo (forvo.com)	Hearing word pronunciation from native speakers	Beginner	Free

How to grow this dictionary

- Add each new resource as a row with the same four columns.
- Add a new table when a purpose or subject area has no home yet, such as music, cooking, or fitness.

- Remove any row that fails the verification check, and note the date it was last checked.
- Keep entries general. The dictionary lists where to learn, never what to learn.